Cracking the Coding Interview

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Introduction

- **The interview**: At most of the top tech companies (and many other companies). algorithm and coding problems form the largest component of the interview process. Think of these as problem-solving questions. The interviewer is looking to evaluate your ability to solve algorithmic problems you haven't seen before.

  Very often, you might only get through one question in an interview. You should do your best to talk out loud throughout the problem and explain your thought process. Your interviewer might jump in sometimes to help you; let them. It's normal and doesn't really mean that you're doing poorly. (That said, of course not needing hints is even better.)

  At the end of the interview, the interviewer will walk away with a gut feel for how you did. A numeric score might be assigned to your performance, but it's not actually a quantitative assessment. There's no chart that says how many points you get for different things. It just doesn't work like that.

  Rather, your interviewer will make an assessment of your performance, usually based on the following:

  - **Analytical skills**: Did you need much help solving the problem? How optimal was your solution? How long did it take you to arrive at a solution? If you had to design/architect a new solution, did you structure the problem well and think through the tradeoffs of different decisions?
  - **Coding skills**: Were you able to successfully translate your algorithm to reasonable code? Was it clean and well-organized? Did you think about potential errors? Did you use good style?
  - **Technical knowledge/ Computer Science fundamentals**: Do you have a strong foundation in computer science and the relevant technologies?
  - **Experience**: Have you made good technical decisions in the past? Have you built interesting, challenging projects? Have you shown drive, initiative, and other important factors?
  - **Culture fit/ Communication skills**: Do your personality and values fit with the company and team? Did you communicate well with your interviewer?

- **Why interview this way?**:

  1. **False negatives are acceptable**: From the company's perspective, it's actually acceptable that some good candidates are rejected. The company is out to build a great set of employees. They can accept that they miss out on some good people. They'd prefer not to, of course, as it raises their recruiting costs. It is an acceptable tradeoff, though, provided they can still hire enough good people.

     They're far more concerned with false positives: people who do well in an interview but are not in fact very good.
  2. **Problem-solving skills are valuable**: If you're able to work through several hard problems (with some help, perhaps), you're probably pretty good at developing optimal algorithms. You're smart.

Smart people tend to do good things, and that's valuable at a company. It's not the only thing that matters, of course, but it is a really good thing.

3. **Basic data structure and algorithm knowledge is useful**: Many interviewers would argue that basic computer science knowledge is, in fact, useful. Understanding trees, graphs, lists, sorting, and other knowledge does come up periodically. When it does, it's really valuable.

   Could you learn it as needed? Sure. But it's very difficult to know that you should use a binary search tree if you don't know of its existence. And if you do know of its existence, then you pretty much know the basics.

   Other interviewers justify the reliance on data structures and algorithms by arguing that it's a good "proxy:' Even if the skills wouldn't be that hard to learn on their own, they say it's reasonably well-correlated with being a good developer. It means that you've either gone through a computer science program (in which case you've learned and retained a reasonably broad set of technical knowledge) or learned this stuff on your own. Either way, it's a good sign.

   There's another reason why data structure and algorithm knowledge comes up: because it's hard to ask problem-solving questions that don't involve them. It turns out that the vast majority of problem-solving questions involve some of these basics. When enough candidates know these basics, it's easy to get into a pattern of asking questions with them.

4. **Whiteboards let you focus on what matters**: It's absolutely true that you'd struggle with writing perfect code on a whiteboard. Fortunately, your interviewer doesn't expect that. Virtually everyone has some bugs or minor syntactical errors.

   The nice thing about a whiteboard is that in some ways, you can focus on the big picture. You don't have a compiler. so you don't need to make your code compile. You don't need to write the entire class definition and boilerplate code. You get to focus on the interesting, "meaty" parts of the code: the function that the question is really all about.

   That's not to say that you should just write pseudocode or that correctness doesn't matter. Most interviewers aren't okay with pseudocode, and fewer errors are better.

   Whiteboards also tend to encourage candidates to speak more and explain their thought process. When a candidate is given a computer, their communication drops substantially

- **How Questions are Selected**: At the vast majority of companies, there are no lists of what interviewers should ask. Rather, each inter viewer selects their own questions

Since it's somewhat of a "free for all" as far as questions, there's nothing that makes a question a "recent Google interview question" other than the fact that some interviewer who happens to work at Google just so happened to ask that question recently.

The questions asked this year at Google do not really differ from those asked three years ago. In fact, the questions asked at Google generally don't differ from those asked at similar companies (Amazon, Face book, etc.).

There are some broad differences across companies. Some companies focus on algorithms (often with some system design worked in), and others really like knowledge-based questions. But within a given category of question, there is little that makes it "belong" to one company instead of another. A Google algorithm question is essentially the same as a Facebook algorithm question.

- **Its all relative**: Interviewers assess you relative to other candidates on that same question by the same interviewer. It's a relative comparison.

  Interview questions are much the same way. Your interviewer develops a feel for your performance by comparing you to other people. It's not about the candidates she's interviewing that week. It's about all the candidates that she's ever asked this question to.

  For this reason, getting a hard question isn't a bad thing. When it's harder for you, it's harder for everyone. It doesn't make it any less likely that you'll do well.

- **Can I re-apply to a company after getting rejected?**: Almost always, but you typically have to wait a bit (6 months to a 1 year). Your first bad interview usually won't affect you too much when you re-interview. Lots of people get rejected from Google or Microsoft and later get offers from them.

- **Behind the scenes**: Once you are selected for an interview, you usually go through a screening interview. This is typically conducted over the phone. College candidates who attend top schools may have these interviews in-person.

  Don't let the name fool you; the "screening" interview often involves coding and algorithms questions, and the bar can be just as high as it is for in-person interviews. If you're unsure whether or not the interview will be technical, ask your recruiting coordinator what position your interviewer holds (or what the interview might cover). An engineer will usually perform a technical interview.

  You typically do one or two screening interviewers before being brought on-site.

  In an on-site interview round, you usually have 3 to 6 in-person interviews. One of these is often over lunch. The lunch interview is usually not technical, and the interviewer may not even submit feedback. This is a good person to discuss your interests with and to ask about the company culture. Your other interviews will be mostly technical and will involve a combination of coding, algorithm, design/architecture, and behavioral/experience questions.

  If you have waited more than a week, you should follow up with your recruiter. If your recruiter does not respond, this does not mean that you are rejected (at least not at any major tech company, and almost any other company). Let me repeat that again: not responding indicates nothing about your status. The intention is that all recruiters should tell candidates once a final decision is made.

  Delays can and do happen. Follow up with your recruiter if you expect a delay, but be respectful when you do. Recruiters are just like you. They get busy and forgetful too.

- **Microsoft**: Microsoft wants smart people. Geeks. People who are passionate about technology. You probably won't be tested on the ins and outs of C ++ AP ls, but you will be expected to write code on the board.

In a typical interview, you'll show up at Microsoft at some time in the morning and fill out initial paper work. You'll have a short interview with a recruiter who will give you a sample question. Your recruiter is usually there to prep you, not to grill you on technical questions. If you get asked some basic technical questions, it may be because your recruiter wants to ease you into the interview so that you're less nervous when the "real" interview starts.

- **Resume**: In the US, it is strongly advised to keep a resume to one page if you have less than ten years of experience. More experienced candidates can often justify 1.5 - 2 pages otherwise.

  Think twice about a long resume. Shorter resumes are often more impressive.

  - Recruiters only spend a fixed amount of time (about 10 seconds) looking at your resume. If you limit the content to the most impressive items, the recruiter is sure to see them. Adding additional items just distracts the recruiter from what you'd really like them to see.
  - Some people just flat-out refuse to read long resumes. Do you really want to risk having your resume tossed for this reason?

  Your resume does not-and should not-include a full history of every role you've ever had. Include only the relevant positions-the ones that make you a more impressive candidate.

  For each role, try to discuss your accomplishments with the following approach: "Accomplished X by implementing Y which led to z: · Here's an example:

  "*Reduced object rendering time by 75% by implementing distributed caching, leading to a 10% reduction in log-in time*"

  Not everything you did will fit into this approach, but the principle is the same: show what you did, how you did it, and what the results were. Ideally, you should try to make the results "measurable" somehow.

- **Projects**: Developing the projects section on your resume is often the best way to present yourself as more experienced. This is especially true for college students or recent grads.

  The projects should include your 2 - 4 most significant projects. State what the project was and which languages or technologies it employed. You may also want to consider including details such as whether the project was an individual or a team project, and whether it was completed for a course or independently. These details are not required, so only include them if they make you look better. Independent projects are generally preferred over course projects, as it shows initiative.

  Do not add too many projects. Many candidates make the mistake of adding all 13 of their prior projects, cluttering their resume with small, non-impressive projects.

  So what should you build? Honestly, it doesn't matter that much. Some employers really like open source projects (it offers experience contributing to a large code base), while others prefer independent projects (it's easier to understand your personal contributions). You could build a mobile app, a web app, or almost anything. The most important thing is that you're building something.

- **Programming Languages and Software**:
  - **Software**: Be conservative about what software you list, and understand what's appropriate for the company. Software like Microsoft Office can almost always be cut. Technical software like Visual Studio and Eclipse is somewhat more relevant, but many of the top tech companies won't even care about that. After all, is it really that hard to learn Visual Studio? Of course, it won't hurt you to list all this software. It just takes up valuable space. You need to evaluate the trade-off of that.
  - **Languages**: Should you list everything you've ever worked with, or shorten the list to just the ones that you're most comfortable with?

    Listing everything you've ever worked with is dangerous. Many interviewers consider anything on your resume to be "fair game" as far as the interview.

    One alternative is to list most of the languages you've used, but add your experience level. This approach is shown below:

    Languages: Java (expert), C ++ (proficient), JavaScript (prior experience).

    Some people list the number of years of experience they have with a particular language, but this can be really confusing. If you first learned Java 10 years ago, and have used it occasionally throughout that time, how many years of experience is this?

    For this reason, the number of years of experience is a poor metric for resumes. It's better to just describe what you mean in plain English.

- **Preparation grid**: Go through each of the projects or components of your resume and ensure that you can talk about them in detail. Filling out a grid like this may help:

Common Questions	Project 1	Project 2	Project 3
Challenges			
Mistakes/Failures			
Enjoyed			
Leadership			
Conflicts			
What You'd Do Differently			

  Along the top, as columns, you should list all the major aspects of your resume, including each project, job, or activity. Along the side, as rows, you should list the common behavioral questions.

  In addition, ensure that you have one to three projects that you can talk about in detail. You should be able to discuss the technical components in depth. These should be projects where you played a central role

- **What are your weaknesses?**: When asked about your weaknesses, give a real weakness! Answers like "My greatest weakness is that I work too hard"tell your interviewer that you're arrogant and/or won't admit to your faults. A good answer conveys a real, legitimate weakness but emphasizes how you work to overcome it.

For example,

"Sometimes, I don't have a very good attention to detail. While that's good because it lets me execute quickly, it also means that I sometimes make careless mistakes. Because of that, I make sure to always have someone else double check my work."

- **What questions should you ask the interviewer?**: Most interviewers will give you a chance to ask them questions. The quality of your questions will be a factor, whether subconsciously or consciously, in their decisions. Walk into the interview with some questions in mind.

  You can think about three general types of questions.

  1. **Genuine Questions**: These are the questions you actually want to know the answers to. Here are a few ideas of questions that are valuable to many candidates:
     (a) "What is the ratio of testers to developers to program managers? What is the interaction like? How does project planning happen on the team?"
     (b) "What brought you to this company? What has been most challenging for you?" These questions will give you a good feel for what the day-to-day life is like at the company.
  2. **Insightful Questions**: These questions demonstrate your knowledge or understanding of technology.
     (a) "I noticed that you use technology X. How do you handle problem Y?"
     (b) the product choose to use the X protocol over the Y protocol? I know it has benefits like A, B, C, but many companies choose not to use it because of issue D:'
  3. **Passion Questions**: These questions are designed to demonstrate your passion for technology. They show that you're interested in learning and will be a strong contributor to the company.
     (a) 'Tm very interested in scalability, and I'd love to learn more about it. What opportunities are there at this company to learn about this?"
     (b) "I'm not familiar with technology X, but it sounds like a very interesting solution. Could you tell me a bit more about how it works?"

     As part of your preparation, you should focus on two or three technical projects that you should deeply master. Select projects that ideally fit the following criteria:

- **Know Your Technical Projects**: As part of your preparation, you should focus on two or three technical projects that you should deeply master. Select projects that ideally fit the following criteria:

  – The project had challenging components (beyond just "learning a lot").
  – You played a central role (ideally on the challenging components).
  – You can talk at technical depth.

  For those projects, and all your projects, be able to talk about the challenges, mistakes, technical decisions, choices of technologies (and tradeoffs of these), and the things you would do differently.

- **Responding to Behavioral Questions**: Behavioral questions allow your interviewer to get to know you and your prior experience better. Remember the following advice when responding to questions.

- **Be specific, not arrogant**: Arrogance is a red flag, but you still want to make yourself sound impressive. So how do you make yourself sound good without being arrogant? By being specific!

  Specificity means giving just the facts and letting the interviewer derive an interpretation. For example, rather than saying that you "did all the hard parts," you can instead describe the specific bits you did that were challenging.
- **Limit details**: When a candidate blabbers on about a problem, it's hard for an interviewer who isn't well versed in the subject or projectto understand it.

  Stay light on details and just state the key points. When possible, try to translate it or at least explain the impact. You can always offer the interviewer the opportunity to drill in further.
- **Focus on yourself, not your team**: Interviews are fundamentally an individual assessment. Unfortunately, when you listen to many candidates (especially those in leadership roles), their answers are about "we'; "us'; and "the team:' The interviewer walks away having little idea what the candidate's actual impact was and might conclude that the candidate did little.

  Pay attention to your answers. Listen for how much you say "we" versus 'T' Assume that every question is about your role, and speak to that.
- **Give structured answers**: There are two common ways to think about structuring responses to a behavioral question: nugget first and S.A.R. These techniques can be used separately or together

- **Nugget first**: Nugget First means starting your response with a "nugget" that succinctly describes what your response will be about.

  For example:

  - **Interviewer**: "Tell me about a time you had to persuade a group of people to make a big change:'
  - **Candidate**: "Sure, let me tell you about the time when I convinced my school to let undergraduates teach their own courses. Initially, my school had a rule where .. :'

- **S.A.R. (Situation, Action, Result)**: The S.A.R. approach means that you start off outlining the situation, then explaining the actions you took, and lastly, describing the result.

  - **Situation**: On my operating systems project, I was assigned to work with three other people. While two were great, the third team member didn't contribute much. He stayed quiet during meetings, rarely chipped in during email discussions, and struggled to complete his components. This was an issue not only because it shifted more work onto us, but also because we didn't know if we could count on him.
  - **Action**: I didn't want to write him off completely yet, so I tried to resolve the situation. I did three things. First, I wanted to understand why he was acting like this. Was it laziness? Was he busy with something else? I struck up a conversation with him and then asked him open-ended questions about how he felt it was going. Interestingly, basically out of nowhere, he said that he wanted to take on the writeup, which is one of the most time intensive parts. This showed me that it wasn't laziness; it was that he didn't feel like he was good enough to write code.

Second, now that I understand the cause, I tried to make it clear that he shouldn't fear messing up. I told him about some of the bigger mistakes that I made and admitted that I wasn't clear about a lot of parts of the project either.

Third and finally, I asked him to help me with breaking out some of the components of the project. We sat down together and designed a thorough spec for one of the big component, in much more detail than we had before. Once he could see all the pieces, it helped show him that the project wasn't as scary as he'd assumed.

- **Result**: With his confidence raised, he now offered to take on a bunch of the smaller coding work, and then eventually some of the biggest parts. He finished all his work on time, and he contributed more in discussions. We were happy to work with him on a future project.

The situation and the result should be succinct. In almost all cases, the "action" is the most important part of the story. Unfortunately, far too many people talk on and on about the situation, but then just breeze through the action.

By using the S.A.R. model with clear situations, actions and results, the interviewer will be able to easily identify how you made an impact and why it mattered

Consider putting your stories in the following grid:

	Nugget	Situation	Action(s)	Result	What It Says
Story 1			1. ... 2. ... 3. ...		
Story 2					

- **Think About What It Says**: What personality attributes has the candidate demonstrated?
  - **Initiative/Leadership**: The candidate tried to resolve the situation by addressing it head-on.
  - **Empathy**: The candidate tried to understand what was happening to the person. The candidate also showed empathy in knowing what would resolve the teammate's insecurity.
  - **Compassion**: Although the teammate was harming the team, the candidate wasn't angry at the team mate. His empathy led him to compassion.
  - **Humility**: The candidate was able to admit to his own flaws (not only to the teammate, but also to the interviewer).
  - **Teamwork/Helpfulness**: The candidate worked with the teammate to break down the project into manageable chunks.

  You should think about your stories from this perspective. Analyze the actions you took and how you reacted. What personality attributes does your reaction demonstrate?

- **So, tell me about yourself...**: Many interviewers kick off the session by asking you to tell them a bit about yourself, or asking you to walk through your resume. This is essentially a"pitch". It's your interviewer's first impression of you, so you want to be sure to nail this.

A typical structure that works well for many people is essentially chronological, with the opening sentence describing their current job and the conclusion discussing their relevant and interesting hobbies outside of work (if any).

1. **Current Role [Headline Only]**: "I'm a software engineer at Microworks, where I've been leading the Android team for the last five years:'
2. **College**: My background is in computer science. I did my undergrad at Berkeley and spent a few summers working at startups, including one where I attempted to launch my own business.
3. **Post College & Onwards**: After college, I wanted to get some exposure to larger corporations so I joined Amazon as a developer. It was a great experience. I learned a ton about large system design and I got to really drive the launch of a key part of AWS. That actually showed me that I really wanted to be in a more entrepreneurial environment.
4. **Current Role [Details]**: One of my old managers from Amazon recruited me out to join her startup, which was what brought me to Microworks. Here, I did the initial system architecture, which has scaled pretty well with our rapid growth. I then took an opportunity to lead the Android team. I do manage a team of three, but my role is primarily with technical leadership: architecture, coding, etc.
5. **Outside of Work**: Outside of work, I've been participating in some hackathons-mostly doing iOS development there as a way to learn it more deeply. I'm also active as a moderator on online forums around Android development.
6. **Wrap Up**: I'm looking now for something new, and your company caught my eye. I've always loved the connection with the user, and I really want to get back to a smaller environment too.

Think carefully about your hobbies. You may or may not want to discuss them.

Often they're just fluff. If your hobby is just generic activities like skiing or playing with your dog, you can probably skip it.

Sometimes though, hobbies can be useful. This often happens when:

- The hobby is extremely unique (e.g., fire breathing). It may strike up a bit of a conversation and kick off the interview on a more amiable note.
- The hobby is technical. This not only boosts your actual skillset, but it also shows passion for technology.
- The hobby demonstrates a positive personality attribute. A hobby like "remodeling your house yourself" shows a drive to learn new things, take some risks, and get your hands dirty (literally and figuratively).

It would rarely hurt to mention hobbies, so when in doubt, you might as well.

- **Sprinkle in Shows of Successes**: In the above pitch, the candidate has casually dropped in some highlights of his background.
  - He specifically mentioned that he was recruited out of Microworks by his old manager, which shows that he was successful at Amazon.
  - He also mentions wanting to be in a smaller environment, which shows some element of culture fit (assuming this is a startup he's applying for).

- He mentions some successes he's had, such as launching a key part of AWS and architecting a scalable system.
- He mentions his hobbies, both of which show a drive to learn.

When you think about your pitch, think about what different aspects of your background say about you. Can you can drop in shows of successes (awards, promotions, being recruited out by someone you worked with, launches, etc.)? What do you want to communicate about yourself?

Technical Questions

- **How to prepare**: Many candidates just read through problems and solutions. That's like trying to learn calculus by reading a problem and its answer. You need to practice solving problems. Memorizing solutions won't help you much.

  For each problem, do the following:

  1. **Try to solve the problem on your own**: Hints are provided at the back of this book, but push yourself to develop a solution with as little help as possible. Many questions are designed to be tough-that's okay! When you're solving a problem, make sure to think about the space and time efficiency.
  2. **Write the code on paper**: Coding on a computer offers luxuries such as syntax highlighting, code completion, and quick debugging. Coding on paper does not. Get used to this-and to how slow it is to write and edit code-by coding on paper.
  3. **Test your code-on paper**: This means testing the general cases, base cases, error cases, and so on. You'll need to do this during your interview, so it's best to practice this in advance.
  4. **Type your paper code as-is into a computer**: You will probably make a bunch of mistakes. Start a list of all the errors you make so that you can keep these in mind during the actual interview.

  The sorts of data structure and algorithm questions that many companies focus on are not knowledge tests. However, they do assume a baseline of knowledge.

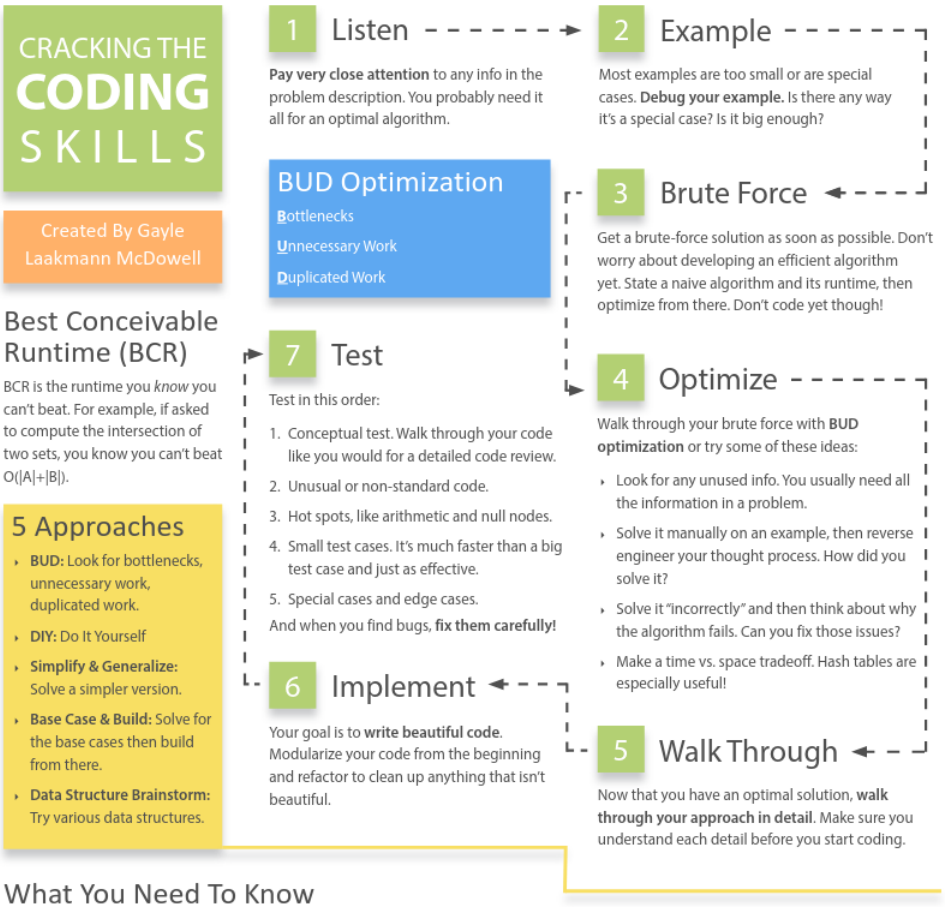
  Most interviewers won't ask about specific algorithms for binary tree balancing or other complex algorithms. Frankly, being several years out of school, they probably don't remember these algorithms either.

  You're usually only expected to know the basics. Here's a list of the absolute, must-have knowledge

Data Structures	Algorithms	Concepts
Linked Lists	Breadth-First Search	Bit Manipulation
Trees, Tries, & Graphs	Depth-First Search	Memory (Stack vs. Heap)
Stacks & Queues	Binary Search	Recursion
Heaps	Merge Sort	Dynamic Programming
Vectors / ArrayLists	Quick Sort	Big O Time & Space
Hash Tables		

  For each of these topics, make sure you understand how to use and implement them and, where applicable, the space and time complexity.

- **A problem solving flowchart**:



- **What to expect**: Interviews are supposed to be difficult. If you don't get every-or any-answer immediately, that's okay! That's the normal experience, and it's not bad.

  Listen for guidance from the interviewer. The interviewer might take a more active or less active role in your problem solving. The level of interviewer participation depends on your performance, the difficulty of the question, what the interviewer is looking for, and the interviewer's own personality.

- **Listen carefully**: For example, suppose a question starts with one of the following lines. It's reasonable to assume that the information is there for a reason.

  – "Given two arrays that are sorted, find .. :"

    You probably need to know that the data is sorted. The optimal algorithm for the sorted situation is probably different than the optimal algorithm for the unsorted situation.

  – "Design an algorithm to be run repeatedly on a server that ..."

    The server/to-be-run-repeatedly situation is different from the run-once situation. Perhaps this means that you cache data? Or perhaps it justifies some reasonable precomputation on the initial dataset?

  Many candidates will hear the problem correctly. But ten minutes into developing an algorithm, some of the key details of the problem have been forgotten. Now they are in a situation where they actually can't solve the problem optimally.

Your first algorithm doesn't need to use the information. But if you find yourself stuck, or you're still working

to develop something more optimal, ask yourself if you've used all the information in the problem. You might even find it useful to write the pertinent information on the whiteboard.

- **State a brute force**: Some candidates don't state the brute force because they think it's both obvious and terrible. But here's the thing: Even if it's obvious for you, it's not necessarily obvious for all candidates. You don't want your interviewer to think that you're struggling to see even the easy solution.

  It's okay that this initial solution is terrible. Explain what the space and time complexity is, and then dive into improvements.

  Despite being possibly slow, a brute force algorithm is valuable to discuss. It's a starting point for optimizations, and it helps you wrap your head around the problem.

- **Optimize**: Once you have a brute force algorithm, you should work on optimizing it. A few techniques that work well are

  1. Look for any unused information. Did your interviewer tell you that the array was sorted? How can you leverage that information?
  2. Use a fresh example. Sometimes, just seeing a different example will unclog your mind or help you see a pattern in the problem.
  3. Solve it"incorrectly:' Just like having an inefficient solution can help you find an efficient solution, having an incorrect solution might help you find a correct solution. For example, if you're asked to generate a random value from a set such that all values are equally likely, an incorrect solution might be one that returns a semi-random value: Any value could be returned, but some are more likely than others. You can then think about why that solution isn't perfectly random. Can you rebalance the probabilities?
  4. Make time vs. space tradeoff. Sometimes storing extra state about the problem can help you optimiz:e the runtime.
  5. Precompute information. Is there a way that you can reorganize the data (sorting, etc.) or compute some values upfront that will help save time in the long run?
  6. Use a hash table. Hash tables are widely used in interview questions and should be at the top of your mind.
  7. Think about the BCR (best conceivable runtime)

- **Walk through**: After you've nailed down an optimal algorithm, don't just dive into coding. Take a moment to solidify your understanding of the algorithm.

  Whiteboard coding is slow-very slow. So is testing your code and fixing it. As a result, you need to make sure that you get it as close to "perfect" in the beginning as possible.

  Walk through your algorithm and get a feel for the structure of the code. Know what the variables are and when they change.

- **Implement**: Now that you have an optimal algorithm and you know exactly what you're going to write, go ahead and implement it.

Start coding in the far top left corner of the whiteboard (you'll need the space). Avoid "line creep" (where each line of code is written an awkward slant). It makes your code look messy and can be very confusing when working in a whitespace-sensitive language, like Python.

Remember that you only have a short amount of code to demonstrate that you're a great developer. Every thing counts. Write beautiful code.

- **Modularized code**: This shows good coding style. It also makes things easier for you. If your algorithm uses a matrix initialized to { { 1, 2, 3}, { 4, 5, 6}, ...}, don't waste your time writing this initialization code. Just pretend you have a function **initIncrementalMatrix(int size)**. Fill in the details later if you need to.
- **Error checks**: Some interviewers care a lot about this, while others don't. A good compromise here is to add a todo and then just explain out loud what you'd like to test.
- **Use other classes/structs where appropriate**: If you need to return a list of start and end points from a function, you could do this as a two-dimensional array. It's better though to do this as a list of **StartEndPair** (or possibly Range) objects. You don't necessarily have to fill in the details for the class. Just pretend it exists and deal with the details later if you have time.
- **Good variable names**: Code that uses single-letter variables everywhere is difficult to read. That's not to say that there's anything wrong with using i and j, where appropriate (such as in a basic for-loop iterating through an array). However, be careful about where you do this. If you write something like

  ```
  0  int i = startOfChild ( array)
  ```

  there might be a better name for this variable, such as startChild.

  Long variable names can also be slow to write though. A good compromise that most interviewers will be okay with is to abbreviate it after the first usage. You can use **startChild** the first time, and then explain to your interviewer that you will abbreviate this as **sc** after this.

The specifics of what makes good code vary between interviewers and candidates, and the problem itself. Focus on writing beautiful code, whatever that means to you.

If you see something you can refactor later on, then explain this to your interviewer and decide whether or not it's worth the time to do so. Usually it is, but not always.

If you get confused (which is common), go back to your example and walk through it again.

- **Test**: You wouldn't check in code in the real world without testing it, and you shouldn't "submit" code in an inter view without testing it either.

There are smart and not-so-smart ways to test your code though.

What many candidates do is take their earlier example and test it against their code. That might discover bugs, but it'll take a really long time to do so. Hand testing is very slow. If you really did use a nice, big example to develop your algorithm, then it'll take you a very long time to find that little off-by-one error at the end of your code.

Instead, try this approach:

1. **Start with a "conceptual" test**: A conceptual test means just reading and analyzing what each line of code does. Think about it like you're explaining the lines of code for a code reviewer. Does the code do what you think it should do?
2. **Weird looking code**: Double check that line of code that says **x = length - 2**. Investigate that for loop that starts at i = 1. While you undoubtedly did this for a reason, it's really easy to get it just slightly wrong.
3. **Hot spots**: You've coded long enough to know what things are likely to cause problems. Base cases in recursive code. Integer division. Null nodes in binary trees. The start and end of iteration through a linked list. Double check that stuff.
4. **Small test cases**: This is the first time we use an actual, specific test case to test the code. Don't use that nice, big 8-element array from the algorithm part. Instead, use a 3 or 4 element array. It'll likely discover the same bugs, but it will be much faster to do so.
5. **Special cases**: Test your code against null or single element values, the extreme cases, and other special cases.

When you find bugs (and you probably will), you should of course fix them. But don't just make the first correction you think of. Instead,carefully analyze why the bug occurred and ensure that your fix is the best one.

- **Look for BUD**:
  - **Bottlenecks**: A bottleneck is a part of your algorithm that slows down the overall runtime. There are two common ways this occurs:
  - **Unnecessary work**:
  - **Duplicated work**:

These are three of the most common things that an algorithm can "waste"time doing. You can walk through your brute force looking for these things. When you find one of them, you can then focus on getting rid of it.

If it's still not optimal, you can repeat this approach on your current best algorithm.

- **DIY (Do it yourself)**: he first time you heard about how to find an element in a sorted array (before being taught binary search), you probably didn't jump to, "Ah ha! We'll compare the target element to the midpoint and then recurse on the appropriate half'

And yet, you could give someone who has no knowledge of computer science an alphabetized pile of student papers and they'll likely implement something like binary search to locate a student's paper. They'll probably say, "Gosh, Peter Smith? He'll be somewhere in the bottom of the stack:'They'II pick a random paper in the middle(ish), compare the name to "Peter Smith'; and then continue this process on the remainder of the papers. Although they have no knowledge of binary search, they intuitively "get it:'

Our brains are funny like this. Throw the phrase "Design an algorithm" in there and people often get all jumbled up. But give people an actual example-whether just of the data (e.g., an array) or of the real-life parallel (e.g., a pile of papers)-and their intuition gives them a very nice algorithm.

I've seen this come up countless times with candidates. Their computer algorithm is extraordinarily slow, but when asked to solve the same problem manually, they immediately do something quite fast. (And it's not too surprisingly, in some sense.Things that are slow for a computer are often slow by hand. Why would you put yourself through extra work?)

Therefore, when you get a question, try just working it through intuitively on a real example. Often a bigger example will be easier.

- **Simplify and generalize**: With Simplify and Generalize, we implement a multi-step approach. First we simplify or tweak some constraint, such as the data type. Then, we solve this new simplified version of the problem. Finally, once we have an algorithm for the simplified problem, we try to adapt it for the more complex version.
- **Base case and build**: With Base Case and Build, we solve the problem first for a base case (e.g., n = 1) and then try to build up from there. When we get to more complex/interesting cases (often n = 3 or n = 4), we try to build those using the prior solutions.

  Base Case and Build algorithms often lead to natural recursive algorithms.
- **Data Structure Brainstorm**: This approach is certainly hacky, but it often works. We can simply run through a list of data structures and try to apply each one. This approach is useful because solving a problem may be trivial once it occurs to us to use, say, a tree